SIZMA TESTİ RAPORU

Rapor Tarihi	29/09/2026
Hazırlayan	Ömer Faruk Baysal
Test Hedefi	PortSwigger Web Academy Lab

1. DOM XSS using web messages

Bu raporda, PortSwigger Web Security Academy'de yer alan DOM XSS message zafiyetlerine ilişkin lablar ele alınmıştır.

2. Yönetici Özeti

Yapılan güvenlik testinde, hedef uygulamanın istemci tarafı (client-side) JavaScript kodunda DOM based Cross-Site Scripting zafiyeti tespit edilmiştir. Uygulama, tarayıcılar arası mesajlaşma için kullanılan postMessage mekanizmasıyla gelen veriyi, gönderenin kaynağını (origin) doğrulamadan ve herhangi bir temizlemeye tabi tutmadan doğrudan bir DOM sink'ine (innerHTML) yazmaktadır. Bu nedenle bir saldırgan, kurbanı hazırladığı kötü niyetli bir sayfaya çekerek, bu sayfadan hedef uygulamaya kötü amaçlı bir mesaj göndermek suretiyle kurbanın tarayıcısında keyfi JavaScript çalıştırabilmektedir. (Durum 1)

Gönderenin kaynağının doğrulanmadığı başka bir durumda (Durum 2) url'in sadece http veya https şemasının url'e eklenerek yönlendirildiği tespit edilmiştir. Bu url'e bu başlıklara ek çalıştırılabilir JavaScript kodu çalıştırılarak zafiyet sömürülebilmektedir.

Yapılan diğer bir testte, hedef uygulamanın postMessage ile gelen veriyi origin doğrulaması yapmadan, oluşturduğu iframe ile birlikte gelen JSON'un url alanı (d.url), şeması (scheme) hiçbir doğrulamaya tabi tutulmadan doğrudan iframe.src sink'ine yazıldığı tespit edilerek DOM tabanlı XSS zafiyetine açık olduğu gözlemlenmiştir. (Durum 3)

Zafiyetler, hazırlanan bir kavram kanıtı (PoC) ile doğrulanmıştır. İstemci tarafında keyfi kod çalıştırılabilmesi, oturum çerezlerinin çalınmasına, kullanıcı adına işlem yapılmasına ve nihayetinde hesabın ele geçirilmesine yol açabileceğinden zafiyetin giderilmesi önerilmektedir.

3. Zafiyet Açıklaması

Durum 1: Web mesajının origin doğrulaması olmadan innerHTML sink'ine yazılması

Hedef uygulamanın ana sayfasında, tarayıcılar arası mesajlaşmayı (web messages) dinleyen bir olay dinleyicisi (event listener) bulunmaktadır. Bu dinleyici, window nesnesine gönderilen 'message' olaylarını yakalamakta ve gelen mesajın içeriğini (e.data) doğrudan sayfadaki bir elemanın innerHTML özelliğine yazmaktadır.

Zafiyeti iki temel eksiklik oluşturmaktadır. Birincisi, dinleyici gelen mesajın kaynağını (e.origin) doğrulamamaktadır; dolayısıyla herhangi bir web sitesi bu sayfaya mesaj gönderebilmektedir. İkincisi, gelen veri herhangi bir temizleme (sanitization) işleminden

geçirilmeden doğrudan innerHTML sink'ine yazıldığından, mesaj içeriği düz metin olarak değil, çalıştırılabilir HTML olarak yorumlanmaktadır.

Bu iki eksikliğin birleşimi, saldırgan kontrolündeki bir kaynağın (postMessage ile gönderilen e.data) tehlikeli bir sink'e (innerHTML) doğrulanmadan akmasına neden olmaktadır. innerHTML ile eklenen <script> etiketleri tarayıcı tarafından çalıştırılmadığından, saldırgan kendiliğinden olay tetikleyen bir HTML öğesi (ör. <img src=x onerror=...>) enjekte ederek kurbanın tarayıcısında keyfi JavaScript çalıştırabilmektedir.

Durum 2: Origin doğrulaması olmayan web uygulamasını JavaScript kodu çalıştırarak yönlendirerek sömürme

Hedef uygulamanın ana sayfasında, tarayıcılar arası mesajlaşmayı (web messages) dinleyen bir olay dinleyicisi (event listener) bulunmaktadır. Bu dinleyici, window nesnesine gönderilen 'message' olaylarını yakalamakta olup gönderenin kaynağını ise doğrulamamaktadır.

Zafiyeti iki temel eksiklik oluşturmaktadır. Birincisi, dinleyici gelen mesajın kaynağını (e.origin) doğrulamamaktadır; dolayısıyla herhangi bir web sitesi bu sayfaya mesaj gönderebilmektedir. İkincisi ise url yönlendirme işleminde sadece http veya https kontrolünün yapılmasıdır. Bu da url'de JavaScript kodu çalıştırılmasına olanak tanımaktadır.

Durum 3: Origin doğrulaması olmadan URL'nin iframe.src sink'ine yazılması

Hedef uygulamanın ana sayfasında, tarayıcılar arası mesajlaşmayı (web messages) dinleyen bir olay dinleyicisi (event listener) bulunmaktadır. Bu dinleyici, window nesnesine gönderilen 'message' olaylarını yakalamakta olup gönderenin kaynağını ise doğrulamamaktadır.

Zafiyeti iki temel eksiklik oluşturmaktadır. Birincisi, dinleyici gelen mesajın kaynağını (e.origin) doğrulamamaktadır; dolayısıyla herhangi bir web sitesi bu sayfaya mesaj gönderebilmektedir. İkincisi, gelen JSON'un url alanı (d.url), şeması (scheme) hiçbir doğrulamaya tabi tutulmadan doğrudan iframe.src sink'ine yazılmaktadır. Bu da javascript: şemalı bir URL ile kurbanın tarayıcısında keyfi JavaScript çalıştırılmasına olanak tanımaktadır.

4. Risk Seviyesi

Risk Seviyesi	Orta
CVSS v3.1 Skoru	6.1
CWE	CWE-79

5. Etki

Zafiyetin başarıyla istismar edilmesi durumunda saldırgan, kurbanın tarayıcısında, hedef uygulamanın kaynağı (origin) bağlamında keyfi JavaScript çalıştırabilmektedir. Saldırı, kurbanın yalnızca saldırgan kontrolündeki bir sayfayı ziyaret etmesiyle gerçekleşmektedir.

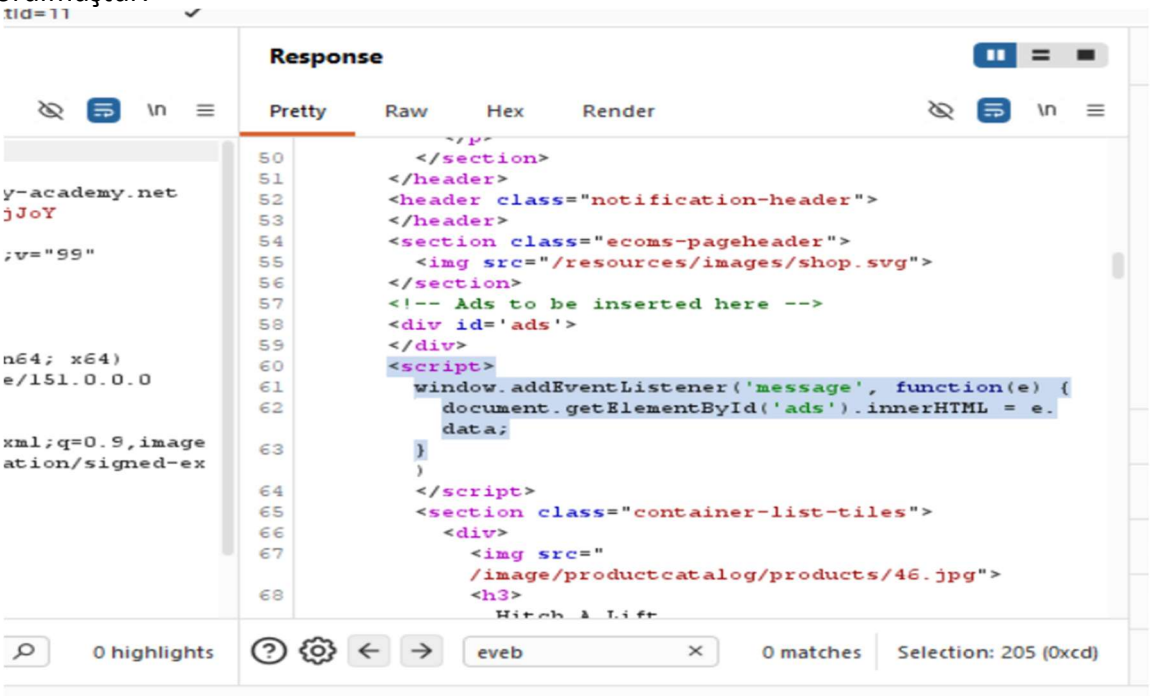
İstemci tarafında keyfi kod çalıştırılabilmesi; uygulama bağlamında erişilebilen oturum çerezlerinin veya diğer hassas verilerin çalınmasına, kurban adına uygulama üzerinde işlem yapılmasına (ör. hesap ayarlarının değiştirilmesi) ve nihayetinde hesabın ele geçirilmesine

olanak tanıyabilir. Etkinin büyüklüğü, çalıştırılan betiğin kapsamına ve uygulamanın niteliğine bağlı olup, kullanıcı verilerinin gizliliği ve bütünlüğü açısından doğrudan risk oluşturur.

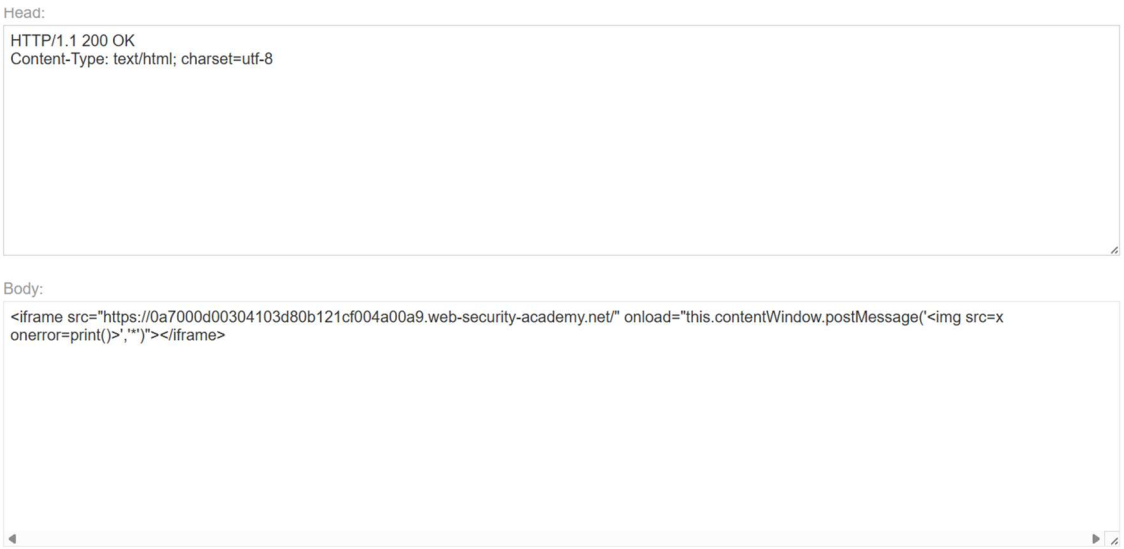
6. Proof Of Concept

Durum 1: Web mesajının origin doğrulaması olmadan innerHTML sink'ine yazılması

Hedef uygulamanın ana sayfasının kaynak kodu incelendiğinde, bir <script> bloğu içinde window.addEventListener('message', ...) ile tanımlanmış bir olay dinleyicisi tespit edilmiştir. Dinleyici incelendiğinde, gelen 'message' olayının içeriğini (e.data) doğrudan ads kimlikli elemanın innerHTML özelliğine yazdığı ve gelen mesajın kaynağını (e.origin) doğrulamadığı görülmüştür.



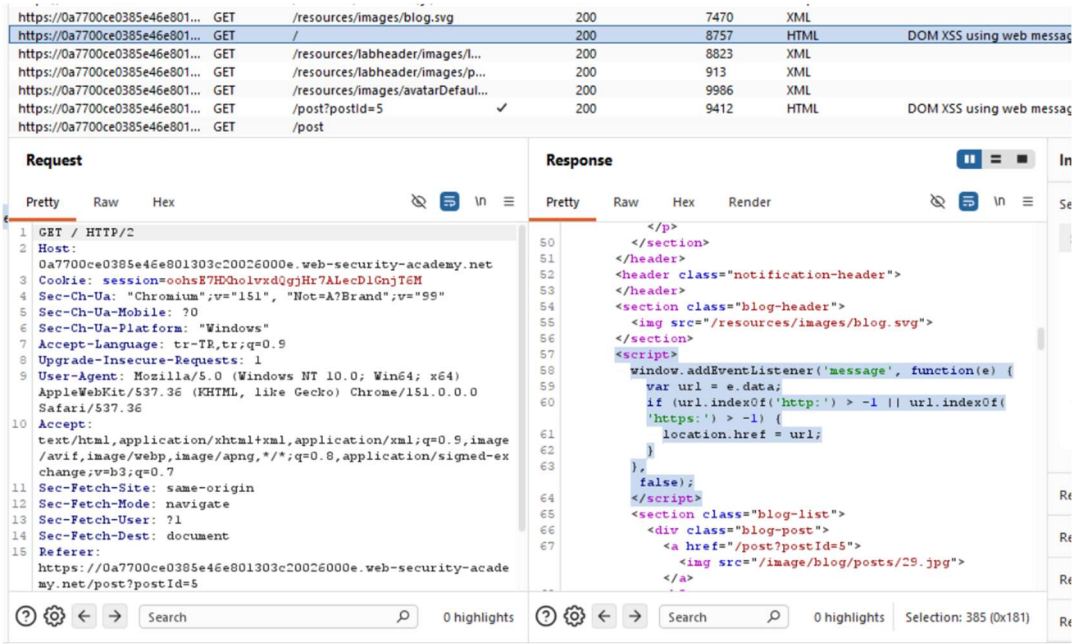
Dinleyici herhangi bir origin doğrulaması yapmadığından, farklı bir kaynaktan gönderilen bir mesajın içeriği de innerHTML'e yazılmaktadır. Aşağıdaki kavram kanıtında hedef sayfa bir <iframe> içine yüklenmekte; sayfa yüklendiğinde (onload) postMessage ile hedef pencereye



kötü amaçlı bir HTML yükü gönderilmektedir. innerHTML ile eklenen <script> etiketleri çalıştırılmadığından, kendiliğinden olay tetikleyen bir <img src=x onerror=...> öğesi kullanılmıştır.

Durum 2: Origin doğrulaması olmayan web uygulamasını JavaScript kodu çalıştırarak yönlendirerek sömürme

Hedef uygulamanın ana sayfasının kaynak kodu incelendiğinde, bir <script> bloğu içinde window.addEventListener('message', ...) ile tanımlanmış bir olay dinleyicisi tespit edilmiştir. Dinleyici incelendiğinde, gelen 'message' olayının içeriğini (e.data) url olarak alan, kontrol eden ve bu urlde http veya https şema değerini kontrol ettiği görülmektedir. Url içinde http veya https var ise location.href (navigasyon sinki) ile yönlendirilmektedir. Ayrıca yine herhangi bir origin kontrolü yer almamaktadır.



Sadece http veya https üzerine bir kontrol yapıldığı için url şemasında oynamalar yapılarak payload yüklenebilmektedir. Dinleyici herhangi bir doğrulama yapmadığı için <iframe> etiketi kullanılarak postMessage ile hedef pencereye kötü amaçlı bir HTML yükü gönderilmektedir.

```
<iframe src="https://0a7700ce0385e46e801303c20026000e.web-security-academy.net/"
 onload="this.contentWindow.postMessage('javascript:print()//https:','*')"></iframe>
```

“javascript:print()//https:” javascript kodu payload olarak iframe içine eklenerek ilgili zafiyet sömürülmektedir. Burada location.href'e javascript:... yazıldığında tarayıcı o kodu çalıştıracaktır. Kontrol olarak sadece http veya https var mı yok mu diye bakıldığı için javascript:print()//https: hem kontrolü geçer (içinde https: var) hem de şema javascript: olduğu için kod çalışacaktır.

Durum 3: Origin doğrulaması olmadan URL'nin iframe.src sink'ine yazılması

Hedef uygulamanın ana sayfasının kaynak kodu incelendiğinde, bir <script> bloğu içinde window.addEventListener('message', ...) ile tanımlanmış bir olay dinleyicisi tespit edilmiştir.

```
<script>
    window.addEventListener('message', function(e) {
        var iframe = document.createElement('iframe'), ACMEplayer = {element: iframe}, d;
        document.body.appendChild(iframe);
        try {
            d = JSON.parse(e.data);
        } catch(e) {
            return;
        }
        switch(d.type) {
            case "page-load":
                ACMEplayer.element.scrollIntoView();
                break;
            case "load-channel":
                ACMEplayer.element.src = d.url;
                break;
            case "player-height-changed":
                ACMEplayer.element.style.width = d.width + "px";
                ACMEplayer.element.style.height = d.height + "px";
                break;
        }
    }, false);
</script>
```

Tıpkı önceki iki durumda olduğu gibi origin kontrolünün olmadığı tespit edilmiştir. İlgili kod incelendiğinde; yeni bir iframe oluşturulduğu, bu iframe'in bir nesneye sarılıp sayfaya eklendiği ve gelen mesajın JSON olarak ayrıştırıldığı gözlemlenmektedir. JSON'daki type alanına bakılarak burada iframe'in src'sine d.url ile bilrikte url yazıldığı tespit edilmiştir. (sink= iframe.src)
Bu iframe'in src'sine javascript: şemalı bir URL yazıldığında JavaScript kodunun çalışacaktır. Sadece bu şemanın JSON olması ve type: "load-channel" olması yeterlidir:

```
{"type":"load-channel","url":"javascript:print()"}
```

Bu JSON payloada eklenecektir ve nihai payload:

```
<iframe src="https://0a1200ff0442b2ee816ced9b0083008f.web-security-academy.net/"
  onload='this.contentWindow.postMessage(`{"type":"load-channel","url":"javascript:print()"}`, "*")'></iframe>
```

Bu payload ile birlikte zafiyet sömürelecektir.

7. Çözüm Önerisi

Mesaj kaynağı (origin) doğrulanmalıdır. postMessage dinleyicisi, gelen mesajın e.origin değerini beklenen ve güvenilen kaynaklara karşı denetlemeli; tanınmayan bir kaynaktan gelen mesajlar işlenmeden reddedilmelidir.

Veri güvenli bir sink'e yazılmalıdır. Gelen veri, HTML olarak yorumlayan innerHTML gibi bir sink'e yazılmamalıdır. Yalnızca metin gerekiyorsa textContent kullanılmalı; HTML gerçekten gerekliyse güçlü bir sanitizasyon kütüphanesi (ör. DOMPurify) ile temizlenmelidir.

Beklenen mesaj yapısı doğrulanmalıdır. Dinleyici, yalnızca beklenen formata ve içeriğe uyan mesajları işlemelidir.

Url yapısında kod çalıştırılmasına karşı önlemler alınmalıdır. Bunun için encoding işlemi yapılabilir, sanitizasyon işlemi ile temizlenmelidir.

Bir navigasyon/kaynak sink'ine (location.href, iframe.src vb.) yazılan değerin şeması denetlenmeli; yalnızca güvenli şemalara (http, https) izin verilmeli, javascript: gibi kod çalıştıran şemalar reddedilmelidir. Mümkünse hedef, izin verilen adreslerden oluşan bir allowlist'e karşı kontrol edilmelidir.

İçerik Güvenliği Politikası (CSP) uygulanmalıdır. Uygun bir Content-Security-Policy ile satır içi (inline) betik ve olay yürütülmesi kısıtlanarak, olası bir enjeksiyonun etkisi savunma derinliği ilkesiyle azaltılmalıdır.

8. Sonuç

Yapılan test sonucunda, hedef uygulamanın postMessage ile gelen veriyi origin doğrulaması yapmadan ve temizlemeden doğrudan innerHTML sink'ine yazması nedeniyle DOM tabanlı XSS zafiyetine açık olduğu tespit ve kavram kanıtı ile doğrulanmıştır. (Durum 1)

Yapılan diğer bir testte, hedef uygulamanın postMessage ile gelen veriyi origin doğrulaması yapmadan ve yönlendirme işleminde url'e yeterli kontrolünün yapılmaması sebebiyle DOM tabanlı XSS zafiyetine açık olduğu tespit ve kavram kanıtı ile doğrulanmıştır. (Durum 2)

Yapılan diğer bir testte, hedef uygulamanın postMessage ile gelen veriyi origin doğrulaması yapmadan, oluşturduğu iframe ile birlikte gelen JSON'un url alanı (d.url), şeması (scheme) hiçbir doğrulamaya tabi tutulmadan doğrudan iframe.src sink'ine yazdığından dolayı DOM tabanlı XSS zafiyetine açık olduğu tespit ve kavram kanıtı ile doğrulanmıştır. (Durum 3)

Zafiyetler, kurbanın tarayıcısında keyfi JavaScript çalıştırılmasına ve buna bağlı olarak oturum veya hesap ele geçirmeye kadar ilerleyebilecek etkisi nedeniyle giderilmesi gereken bir bulgu olarak değerlendirilmiştir.